AEC6 Mini Twitter

Introducción

Este proyecto implementa un sistema tipo red social similar a Twitter usando servicios de AWS. Incluye registro de usuarios, autenticacion, publicacion de comentarios con archivos multimedia y consulta de publicaciones. Este documento explica como funciona el sistema y como realizaria las pruebas del video que pide el enunciado. Todo el codigo esta funcional y listo para desplegarse.

Arquitectura del Sistema

Servicios AWS Utilizados

La arquitectura usa tres servicios principales:

- AWS Lambda - Backend serverless, no necesito gestionar servidores. Cada funcion se ejecuta independientemente.
- Amazon RDS (MySQL) - Base de datos persistente para usuarios y comentarios. Configure una instancia MySQL con las tablas necesarias y datos de prueba.
- Amazon S3 - Dos buckets: uno para hosting de paginas web estaticas y otro para almacenar los videos que suben los usuarios.

Flujo de Datos

El flujo funciona asi:

1. Usuario accede a paginas HTML desde S3 (configurado como sitio web estatico con acceso publico)
2. Las paginas hacen peticiones HTTP directas a las URLs de funciones Lambda
3. Lambda procesa las peticiones, valida datos y consulta/actualiza RDS segun sea necesario
4. Los videos se suben directamente desde el navegador a S3 usando JavaScript
5. Las respuestas vuelven en formato JSON al cliente para mostrar mensajes de exito/error

Estructura del Proyecto

```
AEC6/
├── lambda/
```

```
│   ├── twitterPost/      # Crear comentarios
│   ├── twitterRead/      # Leer comentarios
│   ├── userLogin/        # Autenticar usuarios
│   ├── userRegister/     # Registrar usuarios
│   └── searchByUser/     # Buscar por usuario
├── sql/
│   ├── create_tables.sql # Script creacion tablas
│   └── sample_data.sql   # Datos de prueba
├── web/
│   ├── css/styles.css
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── postComment.html
│   ├── readComment.html
│   └── searchUser.html
├── AEC6_ISMAEL_HERNANDEZ.pdf
└── README.md
```

Base de Datos

Diseño del Esquema

Tengo dos tablas principales:

Tabla `users`

```sql
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_login TIMESTAMP NULL,
    INDEX idx_username (username)
);
```

Por que lo diseñe asi:

- Username unico para evitar duplicados
- Indice en username porque es el campo que mas consulto
- last_login para saber cuando se conecta cada usuario
- password VARCHAR(255) por si en el futuro quiero usar hashes

Tabla `comments`

```sql
CREATE TABLE comments (
    comment_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    username VARCHAR(50) NOT NULL,
    comment_text TEXT NOT NULL,
    video_url VARCHAR(500),
    video_filename VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    likes INT DEFAULT 0,
    INDEX idx_username (username),
    INDEX idx_created_at (created_at DESC),
    FOREIGN KEY (username) REFERENCES users(username)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

Aquí garanticé varias cosas:

- Foreign key con CASCADE para mantener integridad (si borro un usuario, se borran sus comentarios)
- video_url y video_filename opcionales porque no todos los posts tienen video
- Indices en username y created_at para hacer las consultas mas rapidas
- Campo likes por si quiero añadir esa funcionalidad mas adelante

Funciones Lambda

Tengo cinco funciones en Node.js, cada una hace una cosa especifica.

twitterPost - Crear Comentarios

Recibe datos de un comentario y lo guarda en la BD.

Endpoint: POST

Parametros:

```
{
    "username": "string",
    "comment_text": "string",
    "video_url": "string (opcional)",
    "video_filename": "string (opcional)"
}
```

Como funciona:
1. Parseo el body de la peticion
2. Valido que username y comment_text existan
3. Conecto a RDS
4. Inserto usando prepared statements (proteccion contra SQL injection)
5. Devuelvo el ID del comentario creado

twitterRead - Leer Comentarios

Obtiene todos los comentarios ordenados por fecha (mas recientes primero).

Endpoint: GET

Basicamente ejecuto un SELECT con ORDER BY created_at DESC y devuelvo todo en JSON.

userLogin - Autenticar Usuarios

Valida si el usuario y contraseña son correctos.

Endpoint: POST

Parametros:

```
{
    "username": "string",
    "password": "string"
}
```

Como funciona:
1. Busco el usuario en la BD
2. Comparo la contraseña (en texto plano, lo se, no es seguro pero es un proyecto academico)
3. Si coincide, actualizo last_login y devuelvo "OK"
4. Si no coincide, devuelvo "KO"

Segun el enunciado, cuando devuelvo OK el frontend redirige a postComment.html, y cuando devuelvo KO muestra error.

userRegister - Registrar Usuarios

Crea nuevos usuarios.

Endpoint: POST

Parametros:

```
{
    "username": "string",
    "password": "string",
    "email": "string (opcional)"
}
```

Validaciones:
- Username minimo 3 caracteres
- Password minimo 6 caracteres
- Verifico que el username no exista ya

Si todo esta bien devuelvo OK y el frontend redirige a login.html. Si hay algun problema devuelvo KO.

searchByUser - Buscar Comentarios por Usuario

Filtra comentarios de un usuario especifico.

Endpoint: GET con parametro `username` en query string

Es como twitterRead pero con un WHERE username = ? en el SELECT. Esto permite ver el historial completo de posts de cualquier usuario del sistema. La query string la capturo con `event.queryStringParameters.username` y la paso directamente al prepared statement.

Variables de Entorno

Todas las funciones necesitan estas variables configuradas en Lambda:

```
DB_HOST=your-rds-endpoint.region.rds.amazonaws.com
DB_PORT=3306
DB_USER=admin
DB_PASSWORD=your-password
DB_NAME=minitwitter
```

Las configure en la consola de Lambda para cada funcion. Esto es importante porque me permite cambiar la BD (por ejemplo, pasar de desarrollo a produccion) sin tocar el codigo. Ademas es mas seguro que hardcodear las credenciales.

Paginas Web

El proyecto incluye seis paginas HTML que forman la interfaz del usuario:

1. **index.html** - Pagina principal con enlaces de navegacion a todas las funcionalidades
2. **register.html** - Formulario de registro de nuevos usuarios
3. **login.html** - Formulario de autenticacion
4. **postComment.html** - Interfaz para crear comentarios con opcion de adjuntar videos
5. **readComment.html** - Feed que muestra todos los comentarios del sistema
6. **searchUser.html** - Buscador de comentarios filtrados por usuario

Todas las paginas usan JavaScript vanilla para las peticiones HTTP a Lambda mediante fetch(). El diseño esta en un unico archivo styles.css que mantiene consistencia visual en toda la aplicacion.

Partes Avanzadas Implementadas

Servicio web de validacion de usuarios/login

- **Tabla en base de datos:** Cree la tabla `users` con campos username (clave unica), password, email, created_at y last_login. Esta tabla almacena las credenciales de todos los usuarios del sistema.
- **Pagina web:** La pagina `login.html` tiene un formulario con dos campos: username y password. Cuando el usuario hace click en el boton de login, JavaScript captura los datos del formulario y los envia mediante fetch() a la URL de la funcion Lambda userLogin.
- **Funcion Lambda:** La funcion `userLogin` recibe el JSON con username y password. Primero conecto a RDS y busco el usuario en la tabla users con un SELECT. Si el usuario existe, comparo la contraseña recibida con la almacenada. Si coinciden, actualizo el campo last_login a NOW() y devuelvo un JSON con status: “OK”. Si no coinciden o el usuario no existe, devuelvo status: “KO”.
- **Respuesta y redireccion:** En login.html, cuando recibo la respuesta de Lambda, evaluo el campo status. Si es “OK”, uso `window.location.href = 'postComment.html'` para redirigir al usuario a la pagina de crear comentarios. Si es “KO”, muestro un alert con “Usuario o contraseña incorrectos” y el formulario permanece en pantalla para que el usuario reintente.

Servicio web de creacion de usuarios/registro

- **Tabla en base de datos:** Reutilizo la tabla `users` del punto anterior. La misma estructura sirve para almacenar usuarios tanto creados manualmente como registrados desde la web.
- **Pagina web:** La pagina `register.html` contiene un formulario con campos username, password y email (opcional). Incluyo validacion en JavaScript para verificar que username tenga minimo 3 caracteres y password minimo 6 caracteres antes de enviar. Los datos se envian mediante fetch() a la funcion Lambda userRegister.
- **Funcion Lambda:** La funcion `userRegister` recibe username, password y email. Primero valido longitudes minimas. Luego verifico con un SELECT si el username ya existe en la base de datos. Si existe, devuelvo status: "KO" con mensaje "Usuario ya existe". Si no existe, hago un INSERT en la tabla users con los datos recibidos y devuelvo status: "OK".
- **Respuesta y redireccion:** En register.html, si recibo status "OK", redirijo a `login.html` con `window.location.href` para que el usuario inicie sesion con su nueva cuenta. Si recibo "KO", muestro un alert con el mensaje de error (usuario ya existe o datos invalidos) y el usuario puede corregir e intentar de nuevo.

Servicio de busqueda de mensajes por usuario

- **Funcion Lambda:** La funcion `searchByUser` recibe el parametro username a traves de `event.queryStringParameters.username`. Es muy similar a twitterRead pero en lugar de hacer `SELECT * FROM comments`, hago `SELECT * FROM comments WHERE username = ? ORDER BY created_at DESC`. Uso prepared statements para evitar SQL injection. Devuelvo la lista de comentarios en formato JSON.
- **Pagina HTML:** La pagina `searchUser.html` tiene un input de texto donde el usuario escribe el nombre que quiere buscar y un boton. Cuando hace click, JavaScript captura el valor del input y hace un fetch a la URL de Lambda añadiendo el parametro en la query string: `?username=valor_introducido`.
- **Mostrar resultados:** Cuando Lambda devuelve la respuesta JSON con los comentarios, uso JavaScript para crear dinamicamente elementos HTML (divs con clase "comment") y los inserto en el contenedor principal de la pagina. Cada comentario muestra username, texto, fecha y enlace al video si existe. Es practicamente identico al codigo de readComment.html pero filtrado por usuario.

Diseño web

- **Base visual:** Tome como referencia la interfaz de la red social X (Twitter). Use su paleta de colores caracteristica: fondo #15202B (azul oscuro), color primario #1DA1F2 (azul Twitter), y texto claro #E7E9EA para buen contraste.
- **Estructura CSS:** Organice el CSS con variables en `:root` para los colores principales, lo que facilita cambios globales. Todos los elementos comparten un diseño consistente: botones con bordes redondeados, inputs con fondo oscuro y borde sutil, cards para los comentarios con sombra suave.

Pruebas y Validacion

Como perdi acceso a AWS, explico como haría las pruebas del video que pide el enunciado.

1. Crear un nuevo usuario

Mostrar la base de datos antes de crear el usuario:

- Abro MySQL Workbench
- Ejecuto: `SELECT * FROM users;`

Mostrar la base de datos despues de haber creado el usuario:

- Voy a register.html en el navegador
- Relleno el formulario: username “nuevo_usuario”, password “password123”
- Click en registrar
- Vuelvo a MySQL Workbench
- Ejecuto: `SELECT * FROM users`

Mostrar el codigo implementado con explicacion:

- Abro userRegister/index.js
- Primero parseo el body que viene de la pagina web.
- Valido que username tenga minimo 3 caracteres y password minimo 6.
- Luego hago un SELECT para verificar que el username no exista ya en la base de datos. Si no existe, hago un INSERT del nuevo usuario.
- Si todo va bien devuelvo OK y el frontend redirige a login, si hay algun error devuelvo KO.

2. Login de usuario

Mostrar pagina web de login:

- Abro login.html en el navegador
- Muestro el formulario de login

Mostrar la base de datos con los usuarios y contraseñas:

- Voy a MySQL Workbench
- Ejecuto: `SELECT username, password FROM users;`
- Muestro la tabla con todos los usuarios y sus contraseñas

Mostrar el codigo de la funcion lambda:

- Abro userLogin/index.js
- Recibo username y password del formulario.
- Hago un SELECT para buscar ese usuario en la tabla users.
- Si no existe devuelvo KO. Si existe, comparo la contraseña recibida con la almacenada.
- Si coinciden actualizo el campo last_login con NOW() y devuelvo OK, si no coinciden devuelvo KO.
- El frontend esta programado para redirigir a postComment.html cuando recibe OK.

Mostrar que hace login y pasa a postComment.html:

- Vuelvo al navegador en login.html
- Introduzco credenciales: admin / password123

- Click en login
- La pagina redirige automaticamente a postComment.html

3. Demostrar funcionalidad de creacion de comentarios (OBLIGATORIO)

Mostrar tabla de comentarios de base de datos antes del comentario:

- Voy a MySQL Workbench
- Ejecuto: `SELECT * FROM comments ORDER BY created_at DESC LIMIT 5`

Mostrar contenido de bucket S3 antes del comentario:

- Abro la consola de AWS S3
- Navego al bucket de videos
- Muestro los archivos que hay actualmente

Mostrar PostComment.html y realizar un comentario:

- Abro postComment.html en el navegador
- Relleno el formulario:
- Username: admin
- Texto: “Este es un comentario de prueba con video adjunto”
- Selecciono un archivo de video (test.mp4)
- Click en publicar
- Espero a que se suba el video a S3 y se cree el comentario

Mostrar tabla de comentarios despues del comentario:

- Vuelvo a MySQL Workbench
- Ejecuto: `SELECT * FROM comments ORDER BY created_at DESC LIMIT 1`

Mostrar bucket S3 despues del comentario:

- Vuelvo a la consola de AWS S3
- Actualizo el bucket
- Capturo pantalla mostrando que aparece el archivo test.mp4
- El nuevo adjunto deberia estar visible en el bucket

4. Demostrar funcionalidad de readPosts.html (OBLIGATORIO)

Mostrar pagina web readPosts.html, deberia cargar todos los comentarios:

- Abro readComment.html en el navegador
- La pagina carga automaticamente llamando a twitterRead Lambda
- Muestro que aparecen todos los comentarios del sistema con usuario, texto, fecha y enlace al video

Crear un nuevo comentario con postComment.html:

- Voy a postComment.html

- Creo un comentario: username “test_user”, texto “Nuevo comentario para verificar recarga”
- Click en publicar

Volver a readPosts.html y mostrar que aparece ese nuevo comentario:

- Vuelvo a readComment.html
- Recargo la pagina (F5)
- Muestro que el nuevo comentario aparece en el feed, ordenado por fecha (deberia estar arriba)

5. Mostrar busqueda de comentarios por usuario

Desde base de datos, buscar los comentarios del usuario:

- Voy a MySQL Workbench
- Ejecuto: `SELECT * FROM comments WHERE username = 'admin' ORDER BY created_at DESC;`
- Muestro los resultados: todos los comentarios del usuario admin

Mostrar la funcion lambda que realiza la busqueda, y explicar modificaciones:

- Abro searchByUser/index.js
- Esta funcion es muy similar a twitterRead pero con un filtro. Extraigo el parametro username de event.queryStringParameters que viene en la URL como ?username=admin.
- Uso ese valor en un WHERE de la query SQL para filtrar solo los comentarios de ese usuario.
- El resto es igual: devuelvo los resultados en formato JSON.
- Muestro la query SQL en el codigo: `WHERE username = ?`

Mostrar la funcionalidad de busqueda de comentarios en la pagina web:

- Abro searchUser.html en el navegador
- Escribo “admin” en el campo de busqueda
- Click en buscar
- La pagina muestra solo los comentarios del usuario admin
- Comparo visualmente con los resultados de MySQL para verificar que coinciden